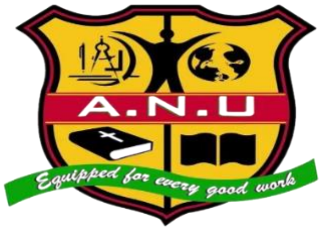
ALL NATIONS UNIVERSITY



UNIVERSITY SHOP INVENTORY TRACKER

A Project Report Submitted By

Omane-Boadi Samuel Jnr

To the

SCHOOL OF ENGINEERING

Submitted to fulfill the requirement for Diploma program

In

Computer Science

DEPARTMENT OF COMPUTER SCIENCE

April 2025

DECLARATION

I hereby declare that the project work entitled "UNIVERSITY SHOP INVENTORY TRACKER" submitted in partial fulfillment of the requirements for the award of DIPLOMA IN COMPUTER SCIENCE, is a record of original work done by me under the supervision and guidance of Dr Theophilus Oware Department of Computer Science. All Nations University Koforidua. This project work has not formed the basis for the award of any Diploma of similar title to any candidate of any university.

…….....

Omane-Boadi Samuel Jnr

……. …….

Head of Department Supervisor

ACKNOWLEDGEMENT

I give thanks to God Almighty for His guidance, protection, and wisdom throughout the course of my studies and the completion of this project. I would also like to express my sincere gratitude to my supervisor for his guidance and valuable support in bringing this project to fruition. Special thanks go to my lecturers, colleagues, and friends who offered encouragement, suggestions, and assistance. Lastly, I acknowledge the unwavering support of my family, whose encouragement and prayers gave me strength to persevere.

ABSTRACT

This project presents the design and implementation of a University Shop Inventory Tracker, a lightweight web-based application for managing and tracking inventory at a university shop. Developed using HTML, CSS, and JavaScript, the application allows users to add, edit, sell, and remove items, track stock levels, and view sales trends through charts. The system uses the browser's localStorage API to persist data, ensuring continuity across sessions without the need for a backend database. A dark mode toggle is also included for improved user experience.

The project demonstrates essential concepts in web development, user interface design, and client-side storage management. It emphasizes usability and performance optimization, ensuring that the system is responsive, accessible, and reliable. The application can serve as a foundation for more advanced inventory systems with server-side integration, authentication, and cloud storage.

TABLE OF CONTENTS

CONTENTS PAGE

LIST OF FIGURES

LIST OF TABLES

CHAPTER ONE

1.0 INTRODUCTION

Inventory control is an essential function for retail operations. Small university shops often rely on manual entry or ad-hoc spreadsheets to track their stock. These approaches are error-prone and lack real-time reporting capabilities. The University Shop Inventory Tracker is a web-based solution developed to provide a lightweight, user-friendly inventory system tailored to the needs of a campus shop. It focuses on essential inventory operations: adding items, editing entries, recording sales, removing obsolete items, and visualizing sales trends with charts.

The project combines modern web development practices with human-centred design principles to ensure that the interface is intuitive for shop attendants while remaining efficient. It demonstrates practical application of HTML, CSS, and JavaScript alongside client-side persistence using the browser's localStorage API.

1.1 Background of the Study

University shops serve students and staff by providing stationery, snacks, apparel, and other essential items. Managing inventory for these shops is a recurring administrative task which, when handled manually, causes frequent stock discrepancies, lost sales opportunities due to stockouts, and inefficient restocking decisions. Automating inventory tasks helps reduce manual effort, minimize human error, and provide faster access to historical sales data that informs procurement.

1.2 Problem Statement

Many university shops in the region still use manual registers or spreadsheets to track stock. These methods are vulnerable to errors: miscounting, double entry, lost paperwork, and inconsistent record-keeping. The lack of an easy-to-use digital tool reduces operational efficiency and makes it difficult to analyze sales trends that could guide purchasing decisions. This project addresses these problems by delivering a simple, modern web application for inventory management.

1.3 Objectives

General Objective: To design and implement a web-based inventory management application suited for a university shop.

Specific Objectives:

- Implement an interface for adding, editing and removing inventory items.
- Implement sales recording (sell button reduces stock and increments sold count).
- Provide search and filter capability for quick item lookup.
- Visualize sales data using charts for top-selling items.
- Ensure data persistence in the browser using localStorage.

1.4 Scope of the Project

The system is a client-side web application. It does not include server-side authentication or remote databases. The scope covers the essential inventory lifecycle within a single browser environment: item management, sales recording, search/filter, CSV export/import, and visualization. The app is suitable for a small university shop operated by a single attendant or small team.

1.5 Significance of the Study

This solution reduces paperwork and errors in inventory tracking, improves restocking decisions through clear visibility of stock levels and sales trends, and serves as a training and demonstrative tool for students learning web development and inventory systems design.

1.6 Motivation

The motivation for this project includes reducing manual workload in small retail settings and building a practical web application that consolidates several core front-end development topics. It also provides a demonstrator for how browser capabilities (like localStorage and Chart.js) can be used to build useful offline-first applications.

CHAPTER TWO

LITERATURE REVIEW AND THEORETICAL FRAMEWORK

2.1 INTRODUCTION TO THE CHAPTER

This chapter reviews the literature and theoretical ideas that inform the design and implementation of the University Shop Inventory Tracker. It synthesizes prior work on inventory management systems, contrasts manual and computerized approaches, examines web usability and client-side persistence (the technical choices made for this project), and outlines the theoretical frameworks that guided system architecture and user-interface decisions. The chapter concludes with a conceptual framework that links user needs, system components and expected outcomes, and a short gap analysis that justifies the project's contribution.

2.2 LITERATURE REVIEW

2.2.1 Inventory Management Systems — overview

Inventory management has been an established area of practice and research in operations and information systems for decades. At its core, inventory management aims to maintain optimal stock levels, minimize stockouts and overstock, and provide accurate records for purchasing, sales, and accounting. Traditional academic treatments emphasize economic order quantity, just-in-time (JIT) methods, and demand forecasting for larger retail operations. However, the needs and constraints of small retail outlets (such as a university shop) differ: they require low cost, ease of use, rapid deployment and minimal maintenance overhead. This difference motivates the development of lightweight, web-based inventory systems that prioritize usability and local, offline-capable persistence rather than complex forecasting features.

2.2.2 Manual versus computerized inventory control

Manual systems (paper ledgers, spreadsheets) are widely used in small shops because they are familiar and require no technical infrastructure. The literature identifies common problems with manual approaches: transcription errors, inconsistent update frequency, lack of consolidated

history, difficulty producing trend reports, and vulnerability to loss or damage. Computerized systems (ranging from desktop applications to cloud solutions) address many of these issues by automating updates, maintaining auditable logs, and enabling analytics. However, full enterprise systems introduce their own challenges for small operators: cost of licensing, hardware and network requirements, complexity, and steep learning curves. Hybrid approaches—simple web apps running entirely in a browser with client-side persistence—offer a middle ground by delivering usability, persistence and reporting without backend infrastructure. Several studies on small business IT adoption highlight that perceived ease-of-use and perceived usefulness are decisive factors for uptake; therefore user-centered design and minimal setup are essential.

2.2.3 Usability principles for small-scale inventory UIs

Human-Computer Interaction (HCI) research emphasises that usability (learnability, efficiency, memorability, error tolerance, and satisfaction) strongly influences whether staff will adopt a tool. Nielsen's heuristics—visibility of system status, match between system and the real world, user control and freedom, consistency, error prevention, recognition versus recall—are widely used as practical guidelines for interface design. For an inventory tracker, this translates into simple forms with required-field hints, inline validation for numeric fields, immediate feedback on actions (e.g., when an item is sold), clear affordances for editing/removing items, and accessible color contrast (including dark mode). Accessibility concerns (keyboard navigation, semantic markup, ARIA attributes) are also important: reports show increased productivity and lower error rates when applications are accessible.

2.2.4 Client-side storage and the localStorage tradeoffs

Modern browsers provide several client-side storage options: cookies, sessionStorage, localStorage, IndexedDB, and the File System Access API. For a small, single-user application that must be easy to deploy and work offline, the Web Storage API—specifically localStorage—offers an appealing balance: it is synchronous, simple to use, persists across sessions, and is widely supported. The literature and documentation point out limitations: storage size limits (typically ~5–10MB per origin), synchronous APIs that can block the main thread for heavy workloads, and lack of structured querying (compared to IndexedDB). For the University Shop

Inventory Tracker, localStorage is suitable because the data per user is small (tens to hundreds of records), operations are lightweight (CRUD on small arrays), and offline persistence without a server is a core requirement. For larger scale or multi-user systems, a backend database and server API would be required; the project design keeps the app architecture modular so such an upgrade is possible later.

2.2.5 Data visualization: role of charts in inventory systems

Visual representations of inventory and sales (bar charts, pie charts, trend lines) are powerful for quick decision making. Research in information visualization demonstrates that well-designed charts improve comprehension and reduce cognitive load compared with raw tables. For this project, Chart.js (a lightweight JavaScript charting library) was selected because it supports responsive canvas-based charts, is easy to integrate into vanilla JavaScript projects, and produces clear bar and pie charts suited to small datasets. The literature recommends that charts used in inventory contexts should: (1) support meaningful aggregation (top sellers, stock by category), (2) allow precise reading (tooltips or labels), and (3) degrade gracefully on small screens—criteria met by Chart.js.

2.2.6 Related work and existing solutions (comparative review)

Several classes of existing solutions exist:

• Spreadsheet templates (Excel / Google Sheets) — low cost, familiar to many users, but error-prone and limited for concurrency and reporting.
• Desktop POS/inventory packages — feature rich but expensive and require installation and maintenance.
• Cloud inventory services (SaaS) — scalable and offer analytics but introduce cost and require user accounts and internet connectivity.
• Lightweight web apps with local persistence — resemble the present project in goals and implementation strategy.

Table 2.1 summarizes key characteristics of each approach and how they compare to the design goals for a university shop.

Table 2.1: Comparison of inventory approaches (textual summary)

— Spreadsheet: Low cost, familiar, weak multi-user support, manual auditing.

— Desktop POS: Feature rich, higher cost, installation overhead.

— Cloud SaaS: Strong analytics, subscription cost, requires internet.

— Local web app (this project): Zero-server deployment, persistent across sessions via localStorage, low cost, single-device focused, easy to use.

2.2.7 Security, privacy and data ownership considerations in client-side apps

Although client-side storage simplifies deployment, it raises privacy and security tradeoffs. localStorage data is stored on the user's device and is scoped to the origin; it is accessible to any script running on that origin. For a local deployment with no third-party scripts, that is acceptable. However, sensitive data (payment details, personal customer information) should never be stored in localStorage. The project avoids collecting personally identifiable customer data, storing only product names, categories, stock and sold counts. For higher-security or multi-user systems, server authentication, TLS, and proper database security controls would be necessary.

2.3 THEORETICAL FRAMEWORK

2.3.1 Human-Computer Interaction (HCI) and Usability Theory

The system design is grounded in HCI and usability theory—primarily in Nielsen's heuristics and Norman's principles of interaction design (affordances, signifiers, feedback). These theories inform concrete UI choices: visible action buttons, immediate feedback when an item is sold, required field validation on add/edit forms, and a simple, consistent layout that aligns with users' mental models (items listed with columns for name, category, stock, sold, and actions). Usability testing (even informal walk-throughs with potential users) is recommended by the literature to discover friction points; the project includes a simple manual test plan to validate typical tasks (add/edit/sell/remove).

2.3.2 CRUD model and client-side application architecture

The application follows the classic CRUD (Create, Read, Update, Delete) model for data

management. In a single-page web app, this model manifests as: input forms for creating and updating items, table rendering for reading, and functions to delete entries. The architecture separates concerns: data storage and retrieval (localStorage wrappers), UI rendering (DOM update functions), and event handling (listeners for user actions). Although not implementing a full MVC framework, the design follows similar separation-of-concern principles to keep the code maintainable and extensible.

2.3.3 Minimalist software engineering principles — KISS and YAGNI

Given the target environment (small university shop), the project follows KISS (Keep It Simple, Stupid) and YAGNI (You Aren't Going to Need It) principles from software engineering: only implement features that directly address the user's needs (add/edit/sell/remove/search/filter/chart/export). Avoid overengineering (e.g., authentication, distributed syncing) in the first iteration; instead design the code so those features can be added later if needed.

2.4 CONCEPTUAL FRAMEWORK

Figure 2.1 (placeholder) depicts the conceptual framework linking user roles, system functions and outcomes:

[Figure 2.1: Conceptual framework — USER → (UI: add/edit/sell/remove/search, Storage: localStorage, Visualization: Chart.js) → OUTCOMES (accurate inventory, sales insight, reduced manual errors)]

Description: The user interacts with a simple UI that executes CRUD operations against a client-side storage model. Visualizations and basic export capabilities provide decision support. The expected outcomes are improved accuracy, faster operations, and better awareness of top-selling items.

2.5 GAP ANALYSIS — WHERE THIS PROJECT FITS

Existing literature and commercial solutions show a gap for small institutions that need a lightweight, zero-backend inventory tool which is:

• Easy to deploy (no server)
• Simple to learn and use (students or shop staff without IT skills)
• Provides immediate visual insights (charts)
• Persists data across sessions (localStorage)
• Affordable (no licence/subscription)

This project specifically targets that niche by providing a browser-based app that requires only a saved HTML file (or a small static hosting) and runs entirely client-side. The project purposely excludes features that would introduce complexity (multi-user concurrency, payment processing) but leaves room for future expansion.

2.6 LIMITATIONS IDENTIFIED IN THE LITERATURE

Studies of client-side applications and small business software adoption note several recurring limitations which the present design acknowledges:

• Single-device limitation: localStorage does not provide automatic syncing between devices.
• Data backup dependency: if the user's device fails and no export was made, data may be lost. The system mitigates this by including export/import functionality (CSV/JSON) for manual backups.
• Security and integrity: without authentication, anyone with access to the device can modify data. The app assumes local device control and recommends best practices (backup, device security) to operators.
• Scalability limits: localStorage size limits and synchronous operations make the approach unsuitable for large inventories or heavy analytic workloads.

2.7 SUMMARY OF THE CHAPTER

This chapter synthesized relevant literature on inventory management, usability and client-side web application design, and established the theoretical principles used in the project (HCI, CRUD architectural separation, and minimalist engineering). It situated the University Shop Inventory Tracker in the landscape of existing solutions and identified a clear niche: a zero-server, low-cost, easy-to-use inventory tool for small institutional shops. The conceptual

framework and gap analysis provide a solid rationale for the implementation choices described in subsequent chapters.

CHAPTER THREE

REQUIREMENT ANALYSIS

3.1 Functional Requirements

Table 3.1: Functional Requirements

Requirement	Description
Add Item	System shall allow adding items with name, category, and stock quantity.
Edit Item	System shall allow editing existing inventory items.
Sell Item	System shall allow selling items and automatically decrement stock.
Remove Item	System shall allow removing items from inventory.
Search/Filter	System shall provide text search and category filtering.
Export/Import	System shall allow exporting inventory to CSV and importing JSON/CSV.

(Table 3.1: Functional Requirements) — place table as shown above and center the caption.

3.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Requirement	Description
Usability	Intuitive and user-friendly interface with clear feedback.
Performance	Fast response for common operations in-browser.
Responsiveness	Works across desktop and mobile devices.

Requirement	Description
Maintainability	Structured codebase and modular functions.
Security	No sensitive data stored; local data only; prevent XSS in inputs.

(Table 3.2: Non-Functional Requirements)

3.3 System Users and Use Cases

List of primary users: Shop attendant, Administrator (future), Auditor (view-only).

Use cases (brief):

- Add new inventory item.
- Edit existing item.
- Sell an item (reduce stock).
- Remove an item.
- Search and filter items.
- Export inventory to CSV.
- Import inventory from JSON/CSV.

CHAPTER FOUR

SYSTEM DESIGN & IMPLEMENTATION

4.1 Architecture Overview

The application is a client-only web app. The browser runs the UI (HTML + CSS), application logic (JavaScript) and stores data in localStorage. Chart.js is included for charts. Minimal architecture:

User (browser) → UI (HTML/CSS/JS) → localStorage
Auxiliary: Chart.js renders charts from in-memory data.

4.2 UI Design

The UI is designed for clarity: header with application name, a table listing items, action buttons (Sell/Edit/Remove), a form for adding items, and a chart for top-selling items. A theme toggle (Light/Dark) is available and persisted to localStorage.

4.3 Implementation Notes

- All event handling uses addEventListener (no inline handlers).
- Data model: each item { id, name, category, stock, sold }.
- Data persistence via JSON in localStorage under a specific key.
- The top-selling chart uses Chart.js; the top 5 items by sold count are displayed.

4.4 Screenshots (placeholders)

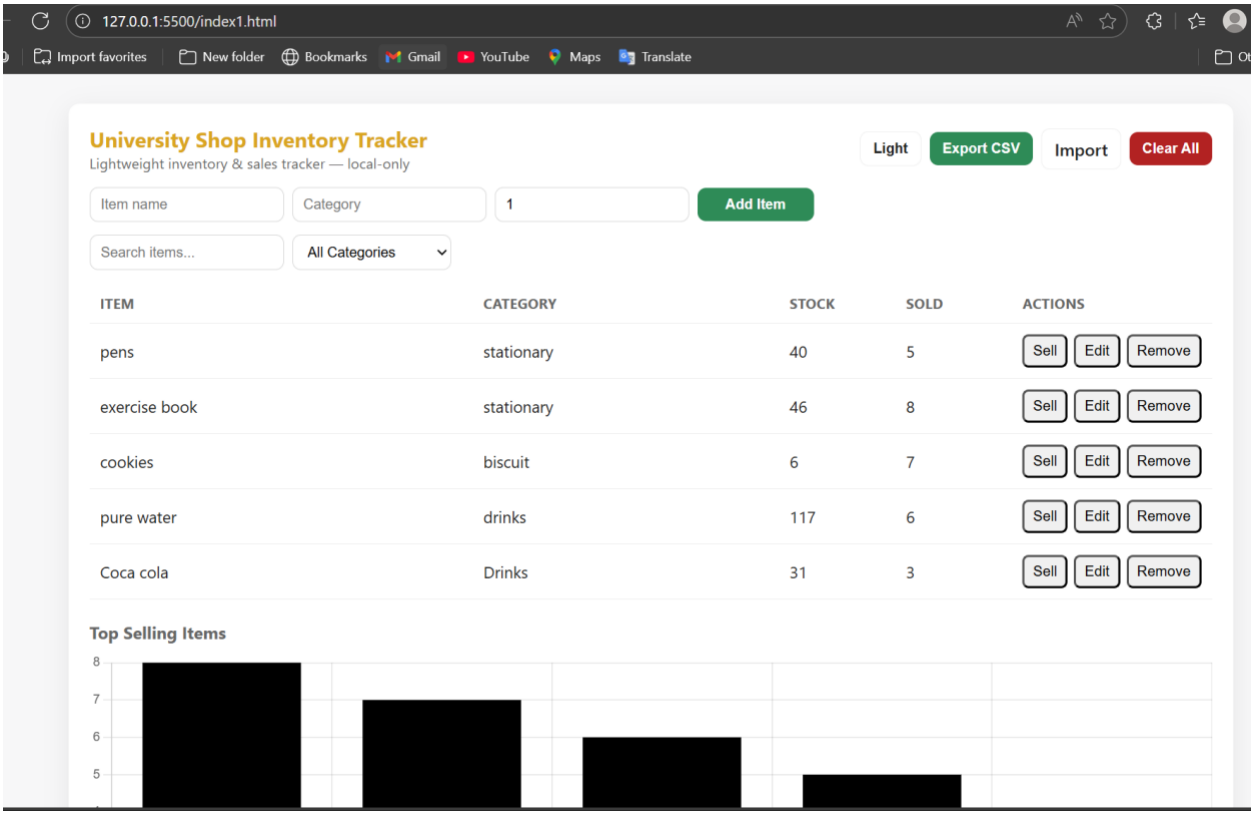


Figure 4.1: Dashboard (Light Mode)

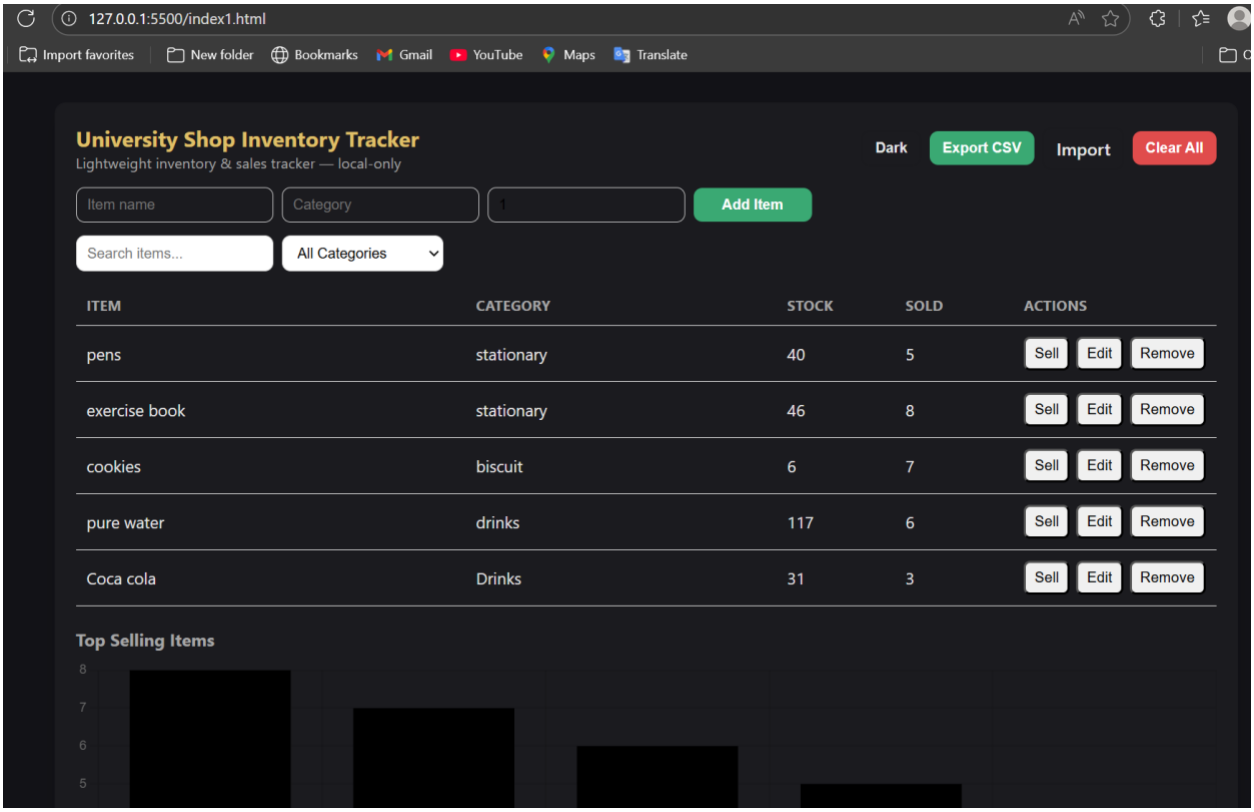


Figure 4.2: Dashboard (Dark Mode)

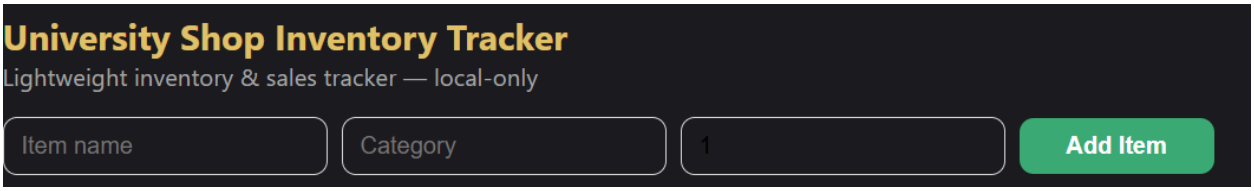


Figure 4.3: Add Item Form

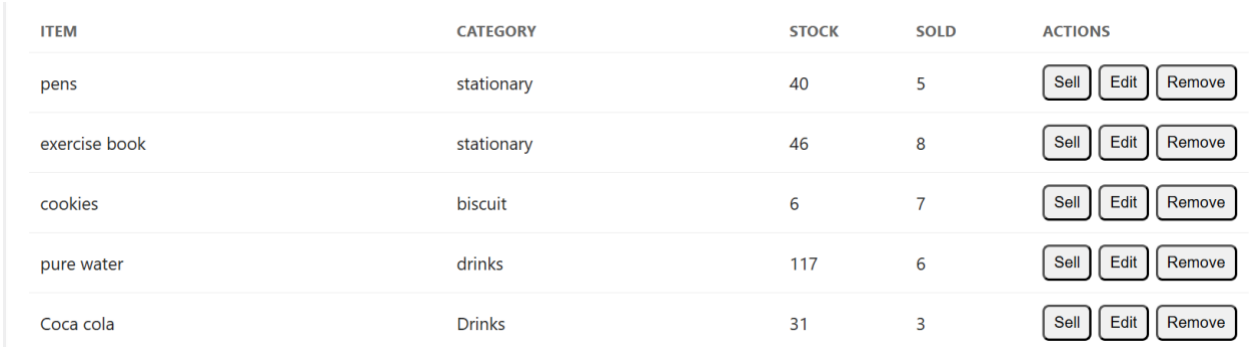


Figure 4.4: Inventory Table

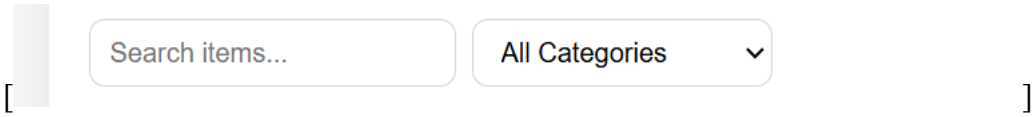


Figure 4.5: Search & Filter

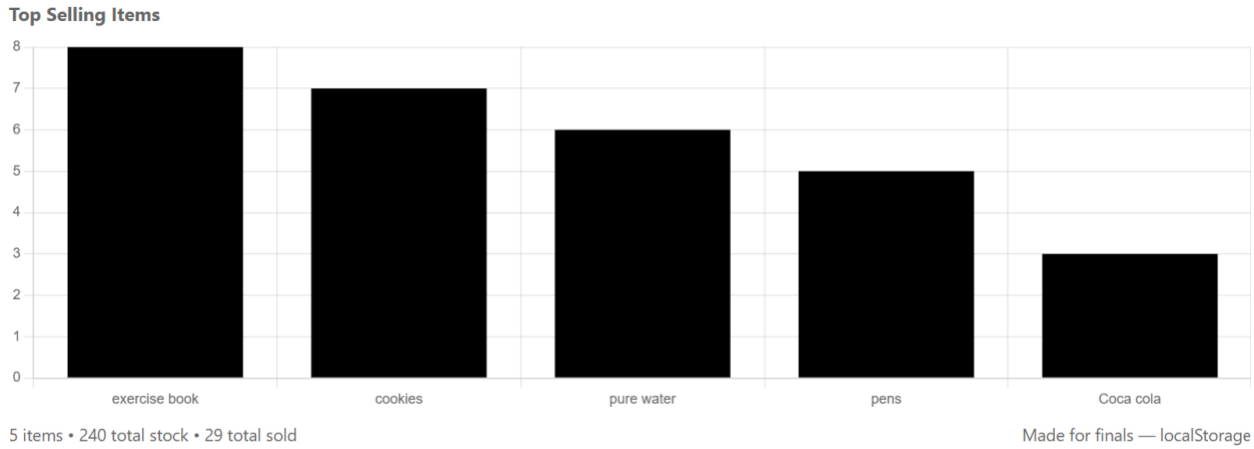


Figure 4.6: Sales Chart

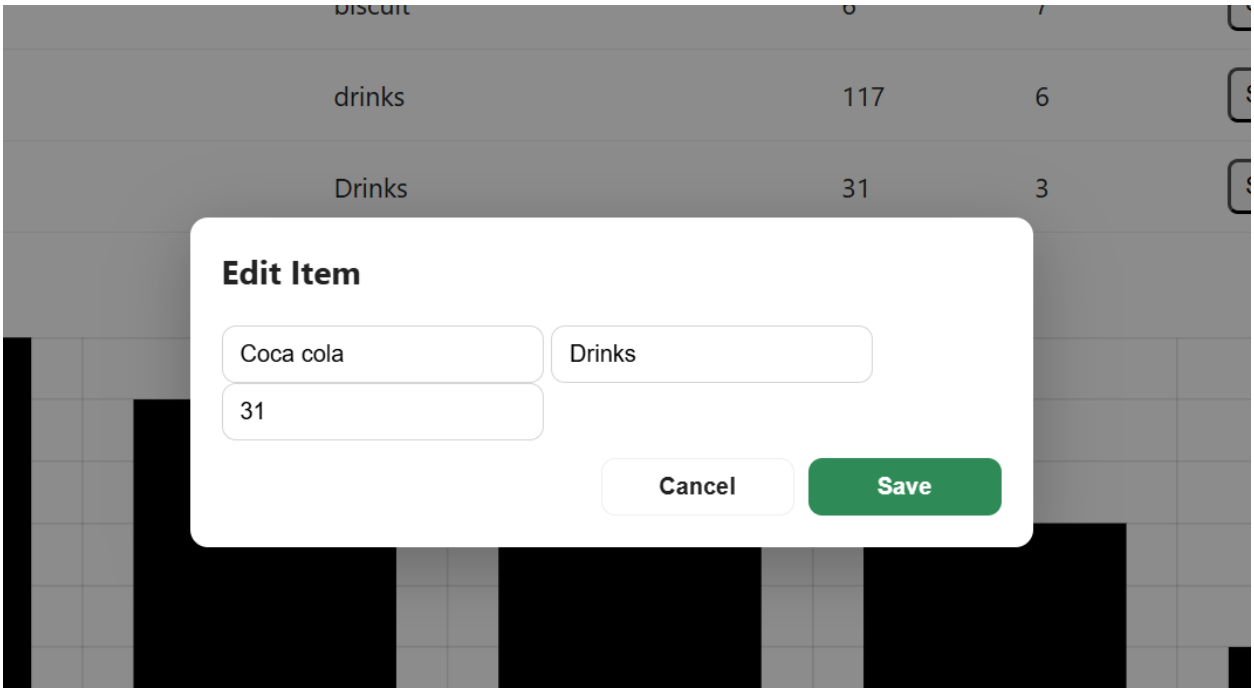


Figure 4.7: Edit Modal

Figure 4.8: Export/Import Dialog

CHAPTER FIVE

TESTING AND EVALUATION

5.1 Testing Strategy

- Functional tests: add item, edit item, remove item, sell item, export/import.
- User acceptance: basic usability checks (labels, button placement).
- Performance: observed instant response for typical inventory sizes (tens to hundreds of items) thanks to in-memory and localStorage operations.
- Cross-browser: tested on modern Chrome/Edge/Firefox desktop.

5.2 Sample Test Cases

Test Case 1: Add Item

- Input: name = Notebook, category = Stationery, stock = 30
- Expected: Item appears in table with stock 30.

Test Case 2: Sell Item

- Precondition: item stock > 0
- Action: Click Sell
- Expected: stock decremented by 1; sold incremented by 1.

Test Case 3: Export/Import

- Action: Export CSV, then import same CSV.
- Expected: imported data matches exported data (IDs preserved where possible).

5.3 Usability Observations

- Buttons labelled “Sell/Edit/Remove” are immediately understandable.
- Search and filter reduce time to find items.

- Dark mode improves readability in low-light conditions.

5.4 Test Evidence (placeholders)

[Insert Figure 5.1: Low Stock Warning here]

Figure 5.1: Low Stock Warning

[Insert Figure 5.2: Confirmation/Alert here]

Figure 5.2: Confirmation/Alert

CHAPTER SIX

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The University Shop Inventory Tracker meets the stated requirements for a simple, reliable, and usable inventory system for small-scale campus shops. It demonstrates how front-end technologies and client-side storage can provide a practical solution with minimal deployment complexity.

6.2 Limitations

- Browser-specific storage (localStorage) limits multi-user and multi-device usage.
- No authentication or role-based access control.
- No scheduled backups or cloud synchronization.

6.3 Future Work

- Add a backend (Node/Express + database) for multi-user access and persistent backups.
- Add user authentication and role-based permissions.
- Add advanced reporting (sales over time, reorder alerts based on thresholds).
- Mobile-specific UI improvements and progressive web app (PWA) packaging.

REFERENCES

1. Jakob Nielsen, Usability Engineering, Morgan Kaufmann, 1994.
2. T. Brinck, D. Gergle, and S. D. Wood, Usability for the Web: Designing Web Sites that Work. Morgan Kaufmann, 2002.
3. Chart.js Documentation — https://www.chartjs.org/docs/latest/
4. MDN Web Docs — JavaScript, HTML, CSS — https://developer.mozilla.org/

APPENDIX A — HTML, CSS AND JAVASCRIPT SOURCE CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>University Shop Inventory Tracker</title>

  <!-- Chart.js CDN -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

  <style>
    :root{
      --gold: #DAA520;
      --green: #2E8B57;
      --red: #B22222;
      --bg: #f7f7f8;
      --card: #ffffff;
      --text: #222;
      --muted: #666;
      --radius: 10px;
```

```
}
.dark {
  --bg: #121217;
  --card: #1b1b1f;
  --text: #e6e6e6;
  --muted: #a7a7a7;
  --gold: #e0c068;
  --green: #3aa973;
  --red: #e04c4c;
}

*{box-sizing: border-box}
body{
  margin:0;
  font-family: Inter, system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial;
  background:var(--bg);
  color:var(--text);
  -webkit-font-smoothing:antialiased;
  -moz-osx-font-smoothing:grayscale;
  padding:28px;
}

.app {
```

```
  max-width:1100px;
  margin:0 auto;
  background:var(--card);
  padding:20px;
  border-radius:var(--radius);
  box-shadow:0 6px 24px rgba(0,0,0,0.07);
}

header{
  display:flex;
  gap:12px;
  align-items:center;
  justify-content:space-between;
  margin-bottom:14px;
}
h1{
  margin:0;
  font-size:1.25rem;
  color:var(--gold);
}
.controls {
  display:flex;
  gap:8px;
```

```
  align-items:center;
}
.btn{
  padding:8px 12px;
  border-radius:8px;
  border:0;
  cursor:pointer;
  background:var(--green);
  color:white;
  font-weight:600;
}
.btn.ghost { background:transparent; color:var(--text); border:1px solid rgba(0,0,0,0.06); }
.btn.warn { background:var(--red); }

.row { display:flex; gap:12px; flex-wrap:wrap; align-items:center; }

form#addForm{
  display:flex;
  gap:8px;
  align-items:center;
  margin-bottom:12px;
  flex-wrap:wrap;
}
```

```
form input, form select {
  padding:8px 10px;
  border-radius:8px;
  border:1px solid #d6d6d6;
  background:transparent;
  min-width:140px;
}
form button { min-width:110px; }

.filters {
  display:flex;
  gap:8px;
  margin-bottom:12px;
  flex-wrap:wrap;
}
.filters input, .filters select {
  padding:8px 10px;
  border-radius:8px;
  border:1px solid #d6d6d6;
  min-width:150px;
}

table {
```

```
  width:100%;
  border-collapse:collapse;
  margin-bottom:12px;
}
th, td {
  padding:10px;
  border-bottom:1px solid #eee;
  text-align:left;
  font-size:0.95rem;
}
th { color:var(--muted); font-weight:700; font-size:0.85rem; text-transform:uppercase; }
tr.low { background: linear-gradient(90deg, rgba(255,0,0,0.03), transparent); }

.actions { display:flex; gap:6px; }
.small { padding:6px 8px; font-size:0.85rem; border-radius:6px; }

.chart-wrap { margin-top:8px; }
canvas { width:100% !important; max-height:320px; }

.footer {
  display:flex;
  justify-content:space-between;
  align-items:center;
```

```
  gap:10px;
  margin-top:10px;
  font-size:0.9rem;
  color:var(--muted);
}

/* Modal */
.modal-backdrop {
  position:fixed; inset:0; display:none; align-items:center; justify-content:center;
  background:rgba(0,0,0,0.45);
}
.modal {
  background:var(--card); padding:18px; border-radius:10px; width:480px; max-width:95%;
  box-shadow:0 10px 40px rgba(0,0,0,0.25);
}
.modal.show { display:flex; }

/* Responsive */
@media (max-width:700px){
  form input, form select { min-width:100%; }
  .filters input, .filters select { min-width:100%; }
  header { flex-direction:column; align-items:flex-start; gap:10px; }
}
```

```
  </style>
</head>
<body>
  <div class="app" id="app">
    <header>
      <div>
        <h1>University Shop Inventory Tracker</h1>
       <div style="font-size:0.85rem; color:var(--muted)">Lightweight inventory & sales tracker — local-only</div>
      </div>

      <div class="controls" role="toolbar" aria-label="toolbar">
            <button id="toggleTheme" class="btn ghost" aria-pressed="false" title="Toggle theme">Toggle Theme</button>
        <button id="exportBtn" class="btn">Export CSV</button>
            <label class="btn ghost" style="display:inline-flex; align-items:center; gap:8px; cursor:pointer;">
          Import
          <input id="importFile" type="file" accept=".json,.csv" style="display:none">
        </label>
        <button id="clearBtn" class="btn warn">Clear All</button>
      </div>
    </header>
```

```
  <!-- Add form -->
  <form id="addForm" aria-label="Add new item">
    <input id="name" type="text" placeholder="Item name" required aria-label="Item name">
    <input id="category" type="text" placeholder="Category" required aria-label="Category">
    <input id="stock" type="number" placeholder="Stock" min="1" value="1" required aria-label="Stock quantity">
    <button type="submit" class="btn">Add Item</button>
  </form>

  <!-- Filters -->
  <div class="filters" role="search" aria-label="Filters">
    <input id="search" type="search" placeholder="Search items..." aria-label="Search items">
    <select id="categoryFilter" aria-label="Filter by category">
      <option value="">All Categories</option>
    </select>
  </div>

  <!-- Inventory Table -->
  <div style="overflow:auto">
    <table aria-live="polite">
      <thead>
```

```
                    <tr><th>Item</th><th>Category</th><th    style="width:110px">Stock</th><th
style="width:110px">Sold</th><th style="width:170px">Actions</th></tr>
      </thead>
      <tbody id="tableBody"></tbody>
    </table>
  </div>

  <!-- Charts -->
  <div class="chart-wrap">
    <h2 style="margin:0 0 8px 0; font-size:1rem; color:var(--muted)">Top Selling Items</h2>
    <canvas id="salesChart" aria-label="Sales chart" role="img"></canvas>
  </div>

  <div class="footer">
    <div id="stats">0 items • 0 total stock • 0 total sold</div>
    <div style="color:var(--muted)">Made for finals — localStorage</div>
  </div>
 </div>

 <!-- Edit Modal -->
  <div  id="modalBackdrop"  class="modal-backdrop"  role="dialog"  aria-modal="true"  aria-
hidden="true">
  <form id="editForm" class="modal" role="form" aria-label="Edit item">
```

```
    <h3 style="margin-top:0">Edit Item</h3>
    <input id="editName" required placeholder="Item name">
    <input id="editCategory" required placeholder="Category">
    <input id="editStock" type="number" min="0" required placeholder="Stock">
    <div style="display:flex; gap:8px; margin-top:10px; justify-content:flex-end;">
      <button type="button" id="closeModal" class="btn ghost">Cancel</button>
      <button type="submit" class="btn">Save</button>
    </div>
  </form>
</div>

<script type="module">
  // ========== Utilities ==========
  const $ = (sel) => document.querySelector(sel);
  const qs = (sel, root=document) => root.querySelector(sel);
  const qsa = (sel, root=document) => Array.from(root.querySelectorAll(sel));
  const debounce = (fn, wait=250) => {
    let t;
    return (...args) => { clearTimeout(t); t = setTimeout(()=>fn(...args), wait); };
  };
  const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2,8);

  // ========== Storage ==========
```

```
const STORAGE_KEY = 'uni_inventory_v1';
const THEME_KEY = 'uni_theme_v1';

function loadInventory(){
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    return raw ? JSON.parse(raw) : [];
  } catch(e) {
    console.error('Failed to parse inventory', e);
    return [];
  }
}
function saveInventory(items){
  localStorage.setItem(STORAGE_KEY, JSON.stringify(items));
}

// ========== State ==========
let inventory = loadInventory();
let chart; // Chart.js instance
let editingId = null;

// ========== DOM refs ==========
const tableBody = $('#tableBody');
```

```
const addForm = $('#addForm');
const nameInput = $('#name');
const categoryInput = $('#category');
const stockInput = $('#stock');
const searchInput = $('#search');
const categoryFilter = $('#categoryFilter');
const statsEl = $('#stats');
const exportBtn = $('#exportBtn');
const importFile = $('#importFile');
const clearBtn = $('#clearBtn');
const toggleThemeBtn = $('#toggleTheme');

// Modal refs
const modalBackdrop = $('#modalBackdrop');
const editForm = $('#editForm');
const editName = $('#editName');
const editCategory = $('#editCategory');
const editStock = $('#editStock');
const closeModalBtn = $('#closeModal');

// ========== Theme ==========
function applyTheme(dark){
  document.documentElement.classList.toggle('dark', !!dark);
```

```
    toggleThemeBtn.setAttribute('aria-pressed', String(!!dark));
    localStorage.setItem(THEME_KEY, JSON.stringify(!!dark));
    toggleThemeBtn.textContent = dark ? 'Light' : 'Dark';
  }
  // init theme
  applyTheme(JSON.parse(localStorage.getItem(THEME_KEY)) || false);
                                toggleThemeBtn.addEventListener('click',          ()          =>
applyTheme(!document.documentElement.classList.contains('dark')));

  // ========== Rendering ==========
  function getCategories(){
    const cats = Array.from(new Set(inventory.map(i => i.category).filter(Boolean)));
    cats.sort((a,b)=>a.localeCompare(b));
    return cats;
  }

  function populateCategoryFilter(){
    const cats = getCategories();
    categoryFilter.innerHTML = `<option value="">All Categories</option>`;
    cats.forEach(c => {
      const opt = document.createElement('option');
      opt.value = c;
      opt.textContent = c;
```

```
    categoryFilter.appendChild(opt);
  });
}

function filteredInventory(){
  const term = searchInput.value.trim().toLowerCase();
  const cat = categoryFilter.value;
  return inventory.filter(item => {
    const matchesTerm = term === '' || item.name.toLowerCase().includes(term) || item.category.toLowerCase().includes(term);
    const matchesCat = !cat || item.category === cat;
    return matchesTerm && matchesCat;
  });
}

function renderTable(){
  tableBody.innerHTML = '';
  const rows = filteredInventory();
  rows.forEach(item => {
    const tr = document.createElement('tr');
    if (item.stock <= 5) tr.classList.add('low');

    // Name cell
```

```
const nameTd = document.createElement('td');
nameTd.textContent = item.name;
tr.appendChild(nameTd);

// Category
const catTd = document.createElement('td');
catTd.textContent = item.category;
tr.appendChild(catTd);

// Stock
const stockTd = document.createElement('td');
stockTd.textContent = item.stock;
tr.appendChild(stockTd);

// Sold
const soldTd = document.createElement('td');
soldTd.textContent = item.sold || 0;
tr.appendChild(soldTd);

// Actions
const actionsTd = document.createElement('td');
const actionsWrap = document.createElement('div');
actionsWrap.className = 'actions';
```

```
// Sell button
const sellBtn = document.createElement('button');
sellBtn.className = 'small small-sell';
sellBtn.textContent = 'Sell';
sellBtn.title = 'Sell 1 unit';
sellBtn.addEventListener('click', () => sellItem(item.id));
actionsWrap.appendChild(sellBtn);

// Edit button
const editBtn = document.createElement('button');
editBtn.className = 'small';
editBtn.textContent = 'Edit';
editBtn.addEventListener('click', () => openEditModal(item.id));
actionsWrap.appendChild(editBtn);

// Remove button
const removeBtn = document.createElement('button');
removeBtn.className = 'small';
removeBtn.textContent = 'Remove';
removeBtn.addEventListener('click', () => removeItem(item.id));
actionsWrap.appendChild(removeBtn);
```

```
    actionsTd.appendChild(actionsWrap);
    tr.appendChild(actionsTd);

    tableBody.appendChild(tr);
  });

  populateCategoryFilter();
  updateStats();
  updateChart();
}

// ========== Actions ==========
function addItem(name, category, stock){
  name = String(name).trim();
  category = String(category).trim();
  stock = Number(stock) || 0;
  if (!name || !category || stock < 0) return false;
  const item = { id: uid(), name, category, stock: Math.max(0, Math.floor(stock)), sold: 0 };
  inventory.unshift(item);
  saveInventory(inventory);
  renderTable();
  return true;
}
```

```
function sellItem(id){
  const idx = inventory.findIndex(i => i.id === id);
  if (idx === -1) return;
  if (inventory[idx].stock <= 0) return;
  inventory[idx].stock -= 1;
  inventory[idx].sold = (inventory[idx].sold || 0) + 1;
  saveInventory(inventory);
  renderTable();
}

function removeItem(id){
  const item = inventory.find(i => i.id===id);
  if (!item) return;
  if (!confirm(`Remove "${item.name}"? This action cannot be undone.`)) return;
  inventory = inventory.filter(i => i.id !== id);
  saveInventory(inventory);
  renderTable();
}

// Edit
function openEditModal(id){
  const item = inventory.find(i => i.id === id);
```

```
  if (!item) return;
  editingId = id;
  editName.value = item.name;
  editCategory.value = item.category;
  editStock.value = item.stock;
  modalBackdrop.style.display = 'flex';
  modalBackdrop.setAttribute('aria-hidden','false');
  editName.focus();
}
function closeEditModal(){
  editingId = null;
  modalBackdrop.style.display = 'none';
  modalBackdrop.setAttribute('aria-hidden','true');
}

// ========== Chart ==========
function updateChart(){
  const ctx = document.getElementById('salesChart').getContext('2d');
  const top = [...inventory].sort((a,b)=> (b.sold||0) - (a.sold||0)).slice(0,6);
  const labels = top.map(i => i.name);
  const data = top.map(i => i.sold || 0);

  if (!chart){
```

```
    chart = new Chart(ctx, {
      type: 'bar',
      data: {
        labels,
        datasets: [{
          label: 'Units sold',
          data,
          backgroundColor: Array(data.length).fill(undefined) // Chart will apply defaults
        }]
      },
      options: {
        responsive: true,
        plugins: { legend: { display: false } },
        scales: { y: { beginAtZero: true, ticks: { precision:0 } } }
      }
    });
  } else {
    chart.data.labels = labels;
    chart.data.datasets[0].data = data;
    chart.update();
  }
}
```

```
// ========== Stats ==========
function updateStats(){
  const totalItems = inventory.length;
  const totalStock = inventory.reduce((s,i)=> s + (Number(i.stock)||0), 0);
  const totalSold = inventory.reduce((s,i)=> s + (Number(i.sold)||0), 0);
 statsEl.textContent = `${totalItems} items • ${totalStock} total stock • ${totalSold} total sold`;
}

// ========== Events ==========
addForm.addEventListener('submit', (e) => {
  e.preventDefault();
  const name = nameInput.value.trim();
  const category = categoryInput.value.trim();
  const stock = Number(stockInput.value);
  if (!name || !category || Number.isNaN(stock) || stock < 0) {
    alert('Please provide valid item name, category and stock.');
    return;
  }
  addItem(name, category, stock);
  addForm.reset();
  nameInput.focus();
});
```

```
  // search & category filter (debounced)
  const onSearch = debounce(() => renderTable(), 220);
  searchInput.addEventListener('input', onSearch);
  categoryFilter.addEventListener('change', renderTable);

  // edit modal events
  closeModalBtn.addEventListener('click', (e) => { e.preventDefault(); closeEditModal(); });
    modalBackdrop.addEventListener('click', (e) => { if (e.target === modalBackdrop) closeEditModal(); });
  editForm.addEventListener('submit', (e) => {
    e.preventDefault();
    if (!editingId) return closeEditModal();
    const item = inventory.find(i => i.id === editingId);
    if (!item) return closeEditModal();
    const newName = editName.value.trim();
    const newCat = editCategory.value.trim();
    const newStock = Number(editStock.value);
    if (!newName || !newCat || Number.isNaN(newStock) || newStock < 0) {
      alert('Please provide valid values.');
      return;
    }
    item.name = newName;
    item.category = newCat;
```

```
  item.stock = Math.floor(newStock);
  saveInventory(inventory);
  renderTable();
  closeEditModal();
});

// Export CSV
function toCSV(items){
  const rows = [['id','name','category','stock','sold']];
  items.forEach(i => rows.push([i.id, i.name, i.category, i.stock, i.sold||0]));
  return rows.map(r => r.map(cell => `"${String(cell).replace(/"/g,'""')}"`).join(',')).join('\n');
}
exportBtn.addEventListener('click', () => {
  const csv = toCSV(inventory);
  const blob = new Blob([csv], { type: 'text/csv' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `inventory_${new Date().toISOString().slice(0,10)}.csv`;
  document.body.appendChild(a);
  a.click();
  a.remove();
  URL.revokeObjectURL(url);
```

```
});

// Import (JSON or CSV)
importFile.addEventListener('change', async (e) => {
  const file = e.target.files[0];
  if (!file) return;
  const txt = await file.text();
  try {
    if (file.name.toLowerCase().endsWith('.json')) {
      const parsed = JSON.parse(txt);
      if (!Array.isArray(parsed)) throw new Error('JSON must be an array of items');
      // Normalize minimal fields
      const parsedItems = parsed.map(it => ({
        id: it.id || uid(),
        name: String(it.name||'Unknown'),
        category: String(it.category||'Uncategorized'),
        stock: Number(it.stock) || 0,
        sold: Number(it.sold) || 0
      }));
      inventory = parsedItems;
      saveInventory(inventory);
      renderTable();
      alert('Imported JSON successfully.');
```

```
    } else if (file.name.toLowerCase().endsWith('.csv')) {
      const lines = txt.split(/\r?\n/).filter(Boolean);
      const parsed = lines.slice(1).map(line => {
        // naive CSV parse that handles quoted fields
        const cells = line.match(/("([^"]|"")*"|[^,]+)/g).map(c => c.replace(/^"|"$/g,'').replace(/""/g,'"'));
        return {
          id: cells[0] || uid(),
          name: cells[1] || 'Unknown',
          category: cells[2] || 'Uncategorized',
          stock: Number(cells[3]) || 0,
          sold: Number(cells[4]) || 0
        };
      });
      inventory = parsed;
      saveInventory(inventory);
      renderTable();
      alert('Imported CSV successfully.');
    } else {
      alert('Unsupported file type (use .json or .csv).');
    }
  } catch(err){
    console.error(err);
```

```
      alert('Failed to import file: ' + err.message);
    } finally {
      importFile.value = '';
    }
  });

  // Clear all
  clearBtn.addEventListener('click', () => {
    if (!confirm('Clear entire inventory? This cannot be undone.')) return;
    inventory = [];
    saveInventory(inventory);
    renderTable();
  });

  // Shortcuts: keyboard N to focus add name
  document.addEventListener('keydown', (e) => {
    if (e.key === 'n' && !e.metaKey && !e.ctrlKey && document.activeElement.tagName !== 'INPUT') {
      e.preventDefault();
      nameInput.focus();
    }
  });
```

```
    // ========== Init ==========
    renderTable();

    // If inventory empty, pre-seed with an example (optional)
    if (inventory.length === 0) {
      // Uncomment to seed example items on first load
      // addItem('Notebook', 'Stationery', 30);
      // addItem('Pen', 'Stationery', 100);
      // addItem('Charger', 'Electronics', 12);
      // addItem('Water Bottle', 'Accessories', 20);
    }
  </script>
</body>
</html>
```